Smart Lost & Found System: A Java Swing and MySQL Campus Application with Rule-Based Matching

Mitesh Nehra
Roll No. 24EARAD102
Dept. of Artificial Intelligence & Data Science
Arya College of Engineering & I.T.
Jaipur, India

Mohit Kumar Sharma
Roll No. 24EARAD103
Dept. of Artificial Intelligence & Data Science
Arya College of Engineering & I.T.
Jaipur, India

Naitik Pathak
Roll No. 24EARAD106
Dept. of Artificial Intelligence & Data Science
Arya College of Engineering & I.T.
Jaipur, India

Paramveer Singh
Roll No. 24EARAD119
Dept. of Artificial Intelligence & Data Science
Arya College of Engineering & I.T.
Jaipur, India

Prabhash Kumar Chaudhary
Roll No. 24EARAD121
Dept. of Artificial Intelligence & Data Science
Arya College of Engineering & I.T.
Jaipur, India

Er. Ram Babu Buri
Supervisor, Assistant Professor
Dept. of Artificial Intelligence & Data Science
Arya College of Engineering & I.T.
Jaipur, India

***Abstract*—Losing identity cards, books, wallets, keys and electronic devices is a routine problem on a college campus. Recovery usually depends on notice boards, word of mouth and messaging groups, so information is scattered, hard to search and quickly forgotten, and there is no structured way to check who owns a recovered item. This paper presents the Smart Lost & Found System, a stand-alone Java Swing application backed by a MySQL database. The system uses a layered architecture of user interface, model, service, Data Access Object (DAO) and JDBC classes, with parameterized SQL for all user input. Two role-specific dashboards are provided: students register, report lost and found items and follow their approval status, while the administrator verifies reports, registers items handed over at the campus desk and reviews claimed items. Every report moves through an explicit status life cycle, and an approved report triggers an in-app notification after which the student accepts the item. A rule-based MatchingService decides whether a lost item and a found item are a possible match using category equality and item-name keyword overlap. We describe the requirements, architecture, database schema, status workflow and matching rule, give worked examples that show where the rule works and fails, and report an honest review of the current implementation, including security and integration gaps. The work shows that a small, explainable, database-backed design can replace informal lost-and-found practices on a campus.**

***Index Terms*—lost and found, campus information system, Java Swing, JDBC, MySQL, Data Access Object, role-based access control, rule-based matching**

I. INTRODUCTION

Every campus has a lost-and-found problem. Students leave identity cards in laboratories, books in libraries and bottles in cafeterias. The usual response is informal: asking friends, checking a notice board, or posting in a class messaging group. These channels have three weaknesses. First, information is fragmented, so a student who lost an item and a student who found it may never meet. Second, old messages cannot be searched, so a report is forgotten within days. Third, ownership is rarely verified, so an item can be handed to the wrong person.

A central, searchable record with a controlled approval process addresses all three problems. We therefore built a desktop application in which students report items, the administrator verifies them, and every record carries a visible status. The application is small enough to be installed on a campus computer, uses only widely available components (Java, Swing, MySQL), and is structured so that undergraduate students can read, test and extend it.

The contributions of this paper are:

1. A layered, DAO-based architecture for a campus lost-and-found application with separate student and administrator interfaces.
2. A status-driven workflow in which reports are verified by an administrator, the student is notified in the application, and the student accepts the item.
3. A transparent rule-based matching method, with worked examples of its true and false matches.
4. A candid review of the implemented system, listing security, integration and testing gaps that must be closed before deployment.

The rest of the paper is organized as follows. Section II gives background. Section III states the requirements and architecture. Section IV describes the implementation. Section V explains the matching rule. Section VI describes the workflow and user interface. Section VII discusses evaluation, Section VIII limitations, Section IX future work, and Section X concludes.

II. BACKGROUND AND RELATED APPROACHES

A. Existing Practice

Campus lost-and-found handling typically uses a physical register at a reception or security desk, notice boards, and social media or messaging groups. A register keeps a record but cannot be searched remotely; messaging groups reach many people but keep no structure and no verification. Commercial and municipal lost-property services use online forms and databases, which is the model that this project adapts to a single campus with a human administrator.

B. Technologies

Java is an object-oriented language, and Swing is its standard toolkit for desktop interfaces. JDBC is the standard API for sending SQL to a relational database [5]. Using a PreparedStatement keeps user data separate from the SQL text, which is the standard defence against SQL injection. MySQL with the InnoDB engine provides primary keys, foreign keys and transactions [6].

C. Design Patterns and Access Control

The Data Access Object pattern moves all persistence code of an entity into one class so that the interface and business logic do not contain SQL [1]. Role-based access control assigns permissions to roles instead of individual users [2]; here the roles are STUDENT and ADMIN.

D. Matching of Short Texts

Lost and found descriptions are short and informal. Substring and keyword overlap are cheap and easy to explain, while edit distance [3] and term weighting [4] are more tolerant. We use the simplest approach in the current version.

III. REQUIREMENTS AND ARCHITECTURE

A. Requirements

The system has sixteen functional requirements, summarized in Table I; the full list follows the structure recommended in [7] and is given in the accompanying Software Requirements Specification.

TABLE I
SUMMARY OF FUNCTIONAL REQUIREMENTS

Group	Requirement
Accounts	Student registration with validation; login with role selection; logout
Reporting	Lost-item and found-item forms with required-field, date and phone validation; status PENDING on save
Administration	Approve or reject lost and found reports; register desk found items; list users and claimed items
Workflow	Notify the student when a report is approved; student accepts or returns the item
Matching	Decide whether a lost item and a found item are a possible match
Persistence	Store all data in MySQL; create the tables on first run

Non-functional requirements are usability, reasonable response time on an ordinary college computer, clear error handling instead of crashes, parameterized SQL, separation of layers and portability on Windows.

B. Architecture

The application has five layers (Fig. 1). The *presentation* layer consists of LoginFrame, RegisterDialog, DashboardFrame and the shared styling class UITheme. The *model* layer holds plain Java classes User, LostItem, FoundItem and Claim with private fields and accessor methods. The *service* layer contains MatchingService with the matching rule. The *data access* layer consists of UserDAO, LostItemDAO, FoundItemDAO and ClaimDAO. The *persistence* layer is DatabaseConnection, a singleton that reads db.properties, opens one JDBC connection to MySQL and creates the tables if they are missing.

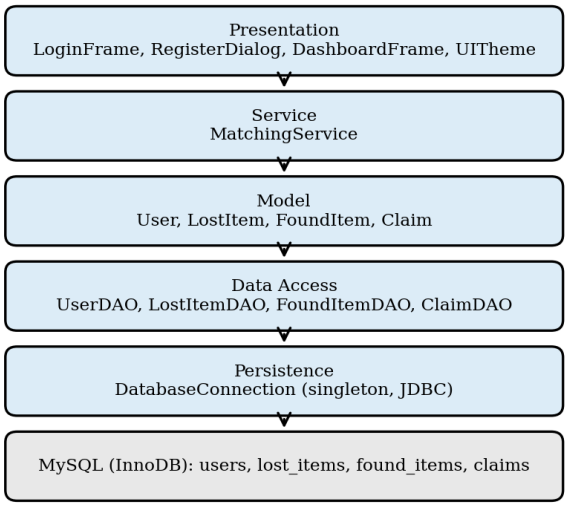


Fig. 1. Layered architecture of the Smart Lost & Found System.

An action in the interface calls a DAO method, which builds a PreparedStatement and returns model objects. The entry point Main sets the look and feel, checks that the database can be reached, shows a warning dialog if it cannot, and opens the login window.

C. Database Design

Four tables are used (Table II). Foreign keys link every report to its owner, and a claim links a found item, a claimant and, optionally, a lost item. All tables use the InnoDB engine and the utf8mb4 character set. The users table holds both roles, distinguished by an ENUM column.

TABLE II
DATABASE TABLES

Table	Key columns	Relationships
users	id (PK), full_name, email (unique), password, role (STUDENT/ADMIN), phone, college_id, created_at	Parent of reports and claims
lost_items	id (PK), user_id (FK), item_name, category, location, lost_date, description, contact_phone, status	ON DELETE CASCADE from users
found_items	id (PK), user_id (FK), item_name, category, found_location, storage_location, found_date, description, contact_phone, status	ON DELETE CASCADE from users
claims	id (PK), lost_item_id (FK, nullable), found_item_id (FK), claimant_id (FK), proof_details, claim_status, admin_remarks, created_at, updated_at	SET NULL on lost item delete; CASCADE on found item and user delete

D. Roles

The login window has a role selector. After authentication the dashboard is built for the account's role. If the chosen role

does not match the account, the user is asked to select the correct one. A student sees four panels: Report Lost Item, Report Found Item, Notifications & Approvals, and My Claimed Items. The administrator sees four panels: Registered Users Details, Register Found Items, Lost Items Approval, and Claimed Items Details.

IV. IMPLEMENTATION

A. Technology and Size

The system is written in Core Java and built with JDK 17. It uses Swing for the interface and MySQL Connector/J as the JDBC driver. The source has about 4,500 lines of Java in 15 files; the largest is DashboardFrame (about 1,800 lines), which contains the panels of both roles. A batch file compiles the code and packages an executable JAR and a Windows launcher.

B. Data Access

Each DAO method follows the same pattern: obtain the connection, prepare a parameterized statement, execute it and map the result set to model objects. For example, authentication in UserDAO is written as a query with two parameters (Listing 1).

```
String sql = "SELECT id, full_name, email,
  password, role, phone, college_id, created_at "
  + "FROM users WHERE LOWER(email) = LOWER(?) "
  + "AND password = ?";
try (PreparedStatement ps =
       conn.prepareStatement(sql)) {
  ps.setString(1, email.trim());
  ps.setString(2, password);
  try (ResultSet rs = ps.executeQuery()) {
    if (rs.next()) { /* map row to User */ }
  }
}
```

Listing 1. Parameterized authentication query in UserDAO.

Because user input is only ever bound as a parameter, text such as a quote or an SQL keyword typed into a form is stored or compared as data and cannot change the statement. Database errors are caught in the DAO, written to the error stream and reported to the interface as a failed operation, which then shows a message dialog instead of closing the program.

C. Input Validation

Validation is done in the interface before a DAO is called. The registration dialog rejects empty fields, a mobile number that is not exactly ten digits, an e-mail without the characters @ and a dot, and unequal passwords; the DAO also checks for an existing e-mail. The report forms require item name, category and location, a date in the format YYYY-MM-DD and a ten-digit contact number. On failure a dialog states what to correct and nothing is saved.

D. Status Life Cycle

Every report has a status. It starts as PENDING, and the administrator changes it to APPROVED or REJECTED. After approval the student gets an in-app notification and can accept the item, which sets the status to CLAIMED and moves the record into the Claimed Items lists. The data layer also supports a longer claim flow (ACTIVE, RETURNED, RESOLVED, CONFIRMED) with admin remarks. Fig. 2 and Table III give the main transitions. The administrator can also enter a found item handed over at the campus desk; it is saved as an approved found item.

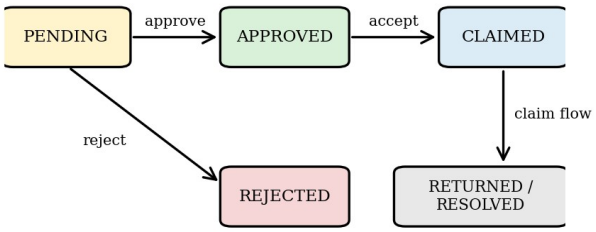


Fig. 2. Main status transitions of a report.

TABLE III
MAIN STATUS TRANSITIONS

From	Event	To
(new)	Student submits report	PENDING
PENDING	Administrator approves	APPROVED
PENDING	Administrator rejects	REJECTED
APPROVED	Student accepts and claims item	CLAIMED
CLAIMED	Receipt confirmed in claim flow	RETURNED (found) / RESOLVED (lost)

E. Start-up and Configuration

Database settings are read from a db.properties file, with defaults for a local MySQL server. On the first connection the application creates the four tables if they are missing and inserts one administrator and five student accounts for demonstration.

V. RULE-BASED MATCHING

A. Rule

The MatchingService compares one lost item with one found item and returns true or false. The rule has two conditions, which must both hold. (1) *Category:* the two categories are equal after trimming and ignoring case; if either category is missing, the result is false. (2) *Name:* both names are non-empty and either name contains the other, or at least one word of the lost item's name with three or more characters occurs inside the found item's name. Comparison ignores case. Algorithm 1 gives the procedure.

Algorithm 1: isMatch(lost, found)

1. if lost = null or found = null then return false
2. if category(lost) = null or category(found) = null then return false
3. if trim(category(lost)) ≠ (ignoring case) trim(category(found)) then return false
4. a ← lower(trim(name(lost))); b ← lower(trim(name(found)))
5. if a is empty or b is empty then return false

6. if a contains b or b contains a then return true
7. for each word w in split(a) do
8. if length(w) ≥ 3 and b contains w then return true
9. return false

The service also provides findMatchesForLostItem, which scans a list of found items and returns those that satisfy the rule, findMatchesForFoundItem for the opposite direction, and getMatchLabel, which returns a text label for display. For a lost item and n found items the scan examines n pairs, and each pair costs time proportional to the lengths of the two names. For campus-sized data this is negligible.

B. Worked Examples

Table IV applies the rule by hand to five pairs of the kind that occur on campus. The examples are illustrative and were derived from the rule itself; they are not measurements on real data.

TABLE IV
WORKED EXAMPLES OF THE MATCHING RULE

Lost item (category)	Found item (category)	Result	Reason
Black Wallet (Wallet / Bag)	Wallet (Wallet / Bag)	Match	Same category; found name is contained in lost name
Student ID Card (Documents)	ID Card (Documents)	Match	Same category; found name is contained in lost name
Black Wallet (Wallet / Bag)	Black Wallet (Other)	No match	Categories differ
Silver Keychain (Keys)	Bunch of Keys (Keys)	No match (missed)	No substring and no shared word of three or more letters
Black Umbrella (Other)	Black Water Bottle (Other)	Match (false)	Shared word “black” in the same category

The third row shows that a wrong category entered by the reporter prevents a match, which is a strict but predictable behaviour. The fourth row shows a false negative caused by different words for a related object. The fifth shows a false positive caused by a common adjective. Both errors are inherent in plain keyword overlap and are the reason that a match is only a suggestion and that final verification is by the administrator.

C. Role of Matching in the System

The rule is intended to give the administrator and the student a short list of candidate records. It does not decide ownership. In the submitted build the service class and the DAO search methods are present, but the dashboard only creates the service object and does not yet display match results (see Section VIII).

VI. WORKFLOW AND USER INTERFACE

A. End-to-End Workflow

1. The user starts the application and logs in, or a new student registers.
2. The role-specific dashboard opens.
3. A student submits a lost or found report; after validation it is saved as PENDING.
4. The administrator opens Lost Items Approval or Register Found Items, inspects the details together with the reporter’s name and contact number, and approves or rejects the report.
5. For an approved report the student sees a notification, opens Notifications & Approvals, and accepts the item or returns it to the administrator.
6. A claimed item appears in My Claimed Items for the student and in Claimed Items Details for the administrator, with the claimant’s name, roll number and mobile number.
7. The user logs out.

B. Interface

The interface uses a dark theme with a left navigation bar, content panels, tables with colour-coded status text, a detail area under each table and action buttons. Lists have a filter box and a Refresh button. The login window offers a show-password option and a link to registration. Screens of the running application are given in the project report (Figures 10.1 to 10.11).

VII. EVALUATION

A. What Has Been Checked

A test plan of sixteen functional cases was prepared. It covers start-up, login for both roles, a wrong password, valid and duplicate registration, lost and found submissions with and without required fields, search, possible-match display, claim submission, administrator status change, persistence after restart, input that looks like SQL, and logout. The source package also contains helper programs for rendering screens and exercising the claim flow.

[Insert results here. For each test case state the actual outcome and Pass or Fail after running it on the final build. Do not report a case as passed unless it has been executed.]

B. Proposed Evaluation of the Matching Rule

The quality of the matching rule has not been measured. We propose the following protocol. Collect or construct 30 to 50 realistic lost reports and the corresponding found reports, label which pairs are true matches, run the rule on all pairs, and report precision, recall and the rank of the true match. If the rule is later extended with scores, report the same measures for different thresholds. Results from such an experiment should be added to this section.

C. Security Review

A review of the source gives the following picture. User input reaches SQL only through PreparedStatement parameters in the DAO classes, which protects against SQL injection. Role separation is achieved by building a different dashboard for each role. However, passwords are stored and compared as plain text, the DAO methods do not check the caller’s role, and predefined accounts with known passwords are created at start-

up. These are acceptable for a classroom demonstration but not for real use.

VIII. LIMITATIONS

1. **Plain-text passwords.** Passwords must be hashed with a salted algorithm such as BCrypt before deployment.
2. **Predefined accounts.** The demonstration administrator and student accounts should be removed or changed for deployment.
3. **Matching is not shown to users.** MatchingService exists and is simple, but the interface does not call it yet.
4. **No match score.** The rule returns true or false. A graded score that combines name, category, location and description would rank candidates better and is a natural extension.
5. **No proof details in the student interface.** The claims table has a proof_details column, but the student claims an approved item with one action. A proof field would help the administrator verify ownership.
6. **No transactions.** A status change that updates two tables is made with separate statements, so a failure between them could leave inconsistent data.
7. **Role checks in the interface only.** An attacker with database access or a modified client is not stopped by the application.
8. **In-app notifications only.** There is no e-mail or SMS message and no audit log of administrator decisions.

IX. FUTURE WORK

- Salted password hashing and a password-reset workflow.
- Calling MatchingService from the student and administrator panels and showing candidate lists.
- A weighted score over name, category, location and description, with a configurable threshold, and tolerant comparison using edit distance [3] or term weighting [4].
- A proof-of-ownership field in the claim form and use of the existing admin_remarks column.
- Transactions around multi-table status changes.
- Item photographs, e-mail notifications and an audit log.
- A client-server or web version so that several campus computers can share one database.
- Automated unit tests for the DAO and service classes.

X. CONCLUSION

This paper presented the Smart Lost & Found System, a desktop application that replaces informal lost-and-found practice on a campus with a central, searchable and reviewable record. Students can register, report lost and found items and follow their status, while the administrator verifies reports, enters items handed over at the campus desk and reviews claimed items. Every record carries a visible status, and an approved report leads to an in-app notification and an accept action by the student.

The system is built from familiar parts: a Java Swing interface, model and service classes, DAO classes and a MySQL database reached through JDBC. This layered DAO design keeps SQL out of the interface, and parameterized statements protect the database from injected input. The rule-based matching method is simple and explainable. Its worked examples show that it finds obvious matches, but also that it can miss related words (keychain and keys) and can match unrelated items that share a common word. For this reason a match is treated only as a suggestion and the administrator remains responsible for verifying ownership.

The review of the implementation shows what remains before real deployment: hashing of passwords, removal of the demonstration accounts, calling the matching service from the screens, a proof-of-ownership field in the claim form, transactions around multi-table updates, and a documented test run with recorded results. Beyond these steps, a weighted match score, photographs, e-mail notifications and a shared client-server version would make the system more accurate and available to the whole campus.

ACKNOWLEDGMENT

The authors thank their project guide and project coordinator for the opportunity and guidance, and their parents and friends for their support in gathering information and finalizing this work.

REFERENCES

[1] D. Alur, J. Crupi, and D. Malks, *Core J2EE Patterns: Best Practices and Design Strategies*. Upper Saddle River, NJ, USA: Prentice Hall, 2001.

[2] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, “Role-based access control models,” *IEEE Computer*, vol. 29, no. 2, pp. 38–47, Feb. 1996.

[3] V. I. Levenshtein, “Binary codes capable of correcting deletions, insertions, and reversals,” *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707–710, 1966.

[4] G. Salton and C. Buckley, “Term-weighting approaches in automatic text retrieval,” *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.

[5] Oracle, “The Java Tutorials: Creating a GUI with Swing” and “JDBC Basics.” [Online]. Available: docs.oracle.com/javase/tutorial

[6] MySQL, “MySQL 8.0 Reference Manual” and “MySQL Connector/J Developer Guide.” [Online]. Available: dev.mysql.com/doc

[7] *IEEE Recommended Practice for Software Requirements Specifications*, IEEE Std 830-1998, 1998.